Manual Técnico de Instalación y Despliegue

Guía para Desarrolladores (GitHub Edition): Instrucciones paso a paso para clonar, configurar la base de datos PostgreSQL, integrar la API de Google Gemini, estructurar el repositorio de forma segura y desplegar el bot 24/7 en Render.

1. Arquitectura y Requisitos Previos

El sistema consta de tres pilares técnicos ejecutados en la nube:

- **Python 3.x & Bot Engine:** Impulsado por `python-telegram-bot` mediante sondeo activo (polling).
- **Base de Datos Relacional:** PostgreSQL alojada en Supabase para persistencia de tareas y actividad de usuarios.
- **Servidor Anti-Sleep (Keep-Alive):** Servidor HTTP integrado ejecutándose en un hilo secundario para evitar que el plan gratuito de Render entre en suspensión por inactividad.

2. Paso 1: Configuración de Telegram (BotFather)

1. Abre Telegram y busca al usuario oficial **@BotFather**.
2. Envía el comando `/newbot` y sigue las instrucciones para nombrar a tu bot.
3. Copia el **HTTP API Token** generado. Esta será tu variable de entorno privada `TELEGRAM_TOKEN`.

3. Paso 2: Base de Datos PostgreSQL (Supabase)

1. Crea un proyecto gratuito en supabase.com y anota tu contraseña segura.
2. Ve a *Project Settings* -> *Database* y copia la URI de conexión. Sustituye la contraseña para formar tu variable `DATABASE_URL`.
3. Ejecuta el siguiente script en el **SQL Editor** de Supabase para inicializar el esquema relacional:

```
CREATE TABLE IF NOT EXISTS tareas (
    id SERIAL PRIMARY KEY,
    chat_id BIGINT NOT NULL,
    motivo TEXT NOT NULL,
    fecha_vencimiento TIMESTAMP WITH TIME ZONE NOT NULL,
    recordatorio_valor INT,
    recordatorio_unidad TEXT,
    estado TEXT DEFAULT 'pendiente',
    es_secuencial BOOLEAN DEFAULT FALSE,
    intervalo_dias INT DEFAULT 0,
    creado_en TIMESTAMP DEFAULT NOW()
);

CREATE TABLE IF NOT EXISTS usuarios_actividad (
    chat_id BIGINT PRIMARY KEY,
    ultimo_login TIMESTAMP WITH TIME ZONE DEFAULT NOW()
);
```

4. Paso 3: Clave de Inteligencia Artificial (Google AI Studio)

Accede a Google AI Studio, genera una clave API y guárdala como `GEMINI_API_KEY` para habilitar el modelo `gemini-2.5-flash` que alimenta el desglose anti-parálisis y el coach de dopamina.

5. Paso 4: Configuración del Repositorio en GitHub

Para asegurar que tu repositorio público o privado en GitHub esté protegido y libre de credenciales hardcodeadas, sube únicamente estos dos archivos en la raíz:

Archivo: `requirements.txt`

```
python-telegram-bot
psycopg2-binary
requests
beautifulsoup4
google-genai
```

Archivo: `bot.py` **(Estructura Base Anonimizada)**

```
import os, time, logging, threading, calendar
from datetime import datetime, timedelta
from http.server import HTTPServer, BaseHTTPRequestHandler
import psycopg2, requests
from google import genai
from telegram import Update, InlineKeyboardButton, InlineKeyboardMarkup
from telegram.ext import Application, CommandHandler, CallbackQueryHandler, MessageHandler, ContextTypes,
filters

logging.basicConfig(format="%(asctime)s - %(levelname)s - %(message)s", level=logging.INFO)
logger = logging.getLogger(__name__)

TOKEN = os.environ.get("TELEGRAM_TOKEN")
DATABASE_URL = os.environ.get("DATABASE_URL")
RENDER_EXTERNAL_URL = os.environ.get("RENDER_EXTERNAL_URL")
GEMINI_API_KEY = os.environ.get("GEMINI_API_KEY")
MI_TELEGRAM_ID = int(os.environ.get("MI_TELEGRAM_ID")) if os.environ.get("MI_TELEGRAM_ID") else None
```

Nota de seguridad: Nunca incluyas tokens ni contraseñas directamente en el código fuente. Utiliza siempre variables de entorno.

6. Paso 5: Despliegue en la Nube (Render.com)

1. Crea un nuevo **Web Service** en Render enlazado a tu repositorio de GitHub.
2. Configura los parámetros de compilación y ejecución:
   - **Build Command:** `pip install -r requirements.txt`
   - **Start Command:** `python bot.py`
   - **Instance Type:** `Free`
3. Configura las siguientes **Environment Variables** en el panel de Render:

TELEGRAM_TOKEN	Token largo proporcionado por BotFather
`DATABASE_URL`	URI completa de conexión a Supabase
`GEMINI_API_KEY`	Clave de Google AI Studio
`MI_TELEGRAM_ID`	Tu ID numérico de Telegram (para privilegios de admin)
`PORT`	`10000`
`RENDER_EXTERNAL_URL`	`https://tu-app.onrender.com`
`WEB_CONCURRENCY`	`1` (evita múltiples hilos de polling)

Haz clic en **Create Web Service**. Render compilará el entorno y en pocos segundos el log mostrará `🚀 Bot activo y escuchando en Telegram`.